Taller de Lógica: Comunicación mediante Tuberías

Malak Sanchez
Monitorias Sistemas Operativos

15 de octubre de 2026

Introducción

Este taller se enfoca en el uso de tuberías anónimas (`pipe`) para el flujo de datos entre procesos. Se explorarán conceptos como la redirección de descriptores de archivos (`dup2`), la ejecución de comandos externos (`exec`) y la comunicación bidireccional.

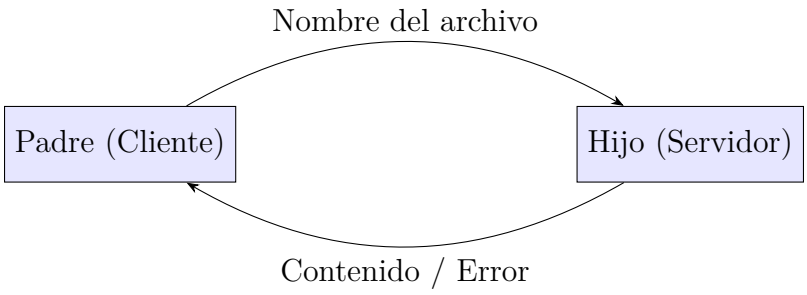
Bloque 1: Implementación de Tuberías Básicas

Utilice fork() y pipe() para implementar flujos de datos entre procesos y redireccionar la salida estándar si es necesario.

Ejercicio 1

Este ejercicio simula un servidor de consultas básico. El proceso padre actúa como cliente y el hijo como servidor.

1. El padre envía un código de producto al hijo mediante una tubería.
2. El hijo consulta el código en un archivo de inventario y envía al padre la cantidad disponible mediante una segunda tubería.
3. El padre imprime la respuesta recibida.
4. **Reto:** el hijo debe responder un estado de error si el código no aparece en el archivo.



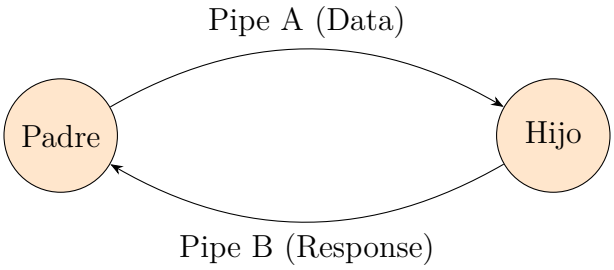
Bloque 2: Comunicación de Datos Dinámicos

Desarrolle programas que intercambien información estructurada entre procesos.

Ejercicio 2

Implemente una comunicación de ïda y vuelta.[en]tre un padre y su hijo utilizando **dos tuberías**.

1. El padre lee una cadena por teclado y la envía por el `Pipe A`.
2. El hijo recibe la cadena, invierte su contenido y la envía por el `Pipe B`.
3. El Padre recibe la respuesta y la imprime en pantalla.



Ejercicio 3

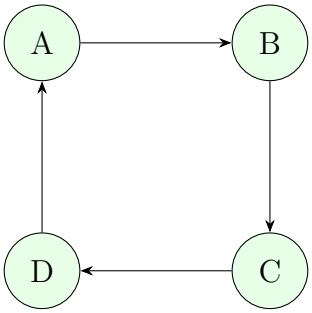
Un proceso padre crea N procesos hijos. Cada hijo debe:

1. Generar una temperatura aleatoria entre 15 y 40.
2. Enviar dicha medición al padre a través de una tubería **compartida** o una tubería por cada hijo.

El padre debe esperar a todos los hijos, recibir las mediciones y calcular el promedio, el máximo y el mínimo.

Ejercicio 4

Cree una cadena cerrada con 4 procesos (A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D $\rightarrow$ A).



Un testigo numérico debe viajar por la cadena. Cada proceso debe duplicar el testigo y pasarlo al siguiente. El programa termina cuando el testigo ha completado 4 vueltas.

Bloque 4: Análisis de Código

Analice el fragmento y responda: ¿Por qué el programa puede bloquearse esperando datos que nunca llegarán?

```
1  int fd[2];
2  pipe(fd);
3  if (fork() == 0) {
4      char buf[10];
5      // El hijo no cierra nada
6      read(fd[0], buf, 10);
7      printf("Hijo leyo: %s\n", buf);
8      exit(0);
9  }
10 // El padre tampoco cierra fd[1] explicitamente antes de esperar
11 write(fd[1], "Hola", 5);
12 wait(NULL);
```